

План:

Этап 1. Фундамент и базовый код

Цель: Научиться логически мыслить, писать простой консольный код, освоить базовую математику и синтаксис.

- **Математический базис:**
 - *Дискретная математика:* теория множеств, логические операторы (AND, OR, NOT).
 - *Теория информации:* системы счисления (двоичная, шестнадцатеричная), биты и байты.
- **Язык программирования (один на выбор):**
 - *Python:* для быстрого старта, веб-разработки или анализа данных.
 - *C++:* для глубокого понимания работы с памятью и железом.
- **Основы программирования:**
 - Управляющие конструкции: циклы (for, while), ветвления (if-else), функции.
 - *Простые структуры:* массивы, строки, динамические списки.
 - *Базовые алгоритмы:* линейный поиск, алгоритмы сортировки (пузырьковая, вставками).
- **Инструменты разработки:**
 - *Командная строка:* основы работы в терминале Bash (Linux/macOS) или PowerShell (Windows).
 - *Git & GitHub:* создание репозитория, коммиты, пуш кода, ветвление.
- **Практический проект:** Написание консольного приложения (например, текстовый менеджер задач, игра «Крестики-нолики» или калькулятор) с сохранением данных в текстовый файл и заливкой кода на GitHub.

Этап 2. Архитектура кода и базы данных

Цель: Перейти от простых скриптов к созданию сложных, правильно структурированных систем с хранением данных.

- **Математический базис:**
 - *Линейная алгебра:* матрицы, векторы, операции над ними.
 - *Математический анализ:* функции, пределы, производные (основа для оптимизации).
- **Продвинутое программирование:**
 - *ООП (Объектно-ориентированное программирование):* инкапсуляция, наследование, полиморфизм.
 - Принципы проектирования *SOLID* и чистый код.
 - *Сложные структуры данных:* стеки, очереди, хеш-таблицы, бинарные деревья, графы.
 - *Сложные алгоритмы:* бинарный поиск, быстрые сортировки, обход графов (в ширину/глубину), оценка сложности *Big O*.
- **Базы данных (СУБД):**
 - *Реляционные БД:* основы реляционной модели, PostgreSQL или MySQL.
 - *Язык SQL:* написание запросов (SELECT, JOIN, GROUP BY), индексы, нормализация таблиц.
- **Инженерия качества:**
 - *Тестирование:* написание автоматических Unit-тестов для проверки логики кода.
- **Практический проект:** Разработка бэкенд-части (API) для библиотеки или интернет-магазина. Код должен быть написан по ООП, протестирован юнит-тестами и интегрирован с базой данных PostgreSQL.

Этап 3. Специализация и инфраструктура

Цель: Определить направление (Web/Mobile) и научиться разворачивать свои приложения в сети, обеспечивая их безопасность.

- **Математический базис:**
 - *Теория вероятностей:* случайные величины, распределения, основы статистики.
- **Направление разработки (выбрать одно):**
 - *Веб-разработка (Фронтенд):* HTML5, CSS3, JavaScript/TypeScript, основы React или Vue.

- Веб-разработка (Бэкенд): Node.js, Django или Spring.
- Мобильная разработка: Kotlin (Android) или Swift (iOS).
- **Сети и Инфраструктура:**
 - Сети: стек TCP/IP, протоколы HTTP/HTTPS, DNS.
 - Архитектура обмена: проектирование и интеграция REST API, форматы данных JSON/XML.
 - Контейнеризация: Docker (сборка приложений в изолированные контейнеры).
- **Информационная безопасность:**
 - Аутентификация: организация входа в систему через JWT-токены или OAuth.
 - Защита приложений: изучение уязвимостей из OWASP Top 10 (SQL-инъекции, XSS) и защита от них.
- **Практический проект:** Создание полноценного веб- или мобильного приложения (например, мини-соцсеть или сервис доставки еды) с разделением на фронтенд и бэкенд, авторизацией пользователей, упаковкой в Docker и защитой от базовых атак.

Этап 4. Промышленные стандарты и командная работа

Цель: Подготовка к коммерческой разработке (уровень Junior/Middle), масштабирование систем и работа по гибким методологиям.

- **Архитектурные паттерны:**
 - Микросервисная архитектура, паттерны MVC / MVVM.
 - Проектирование распределенных и высоконагруженных систем.
- **Продвинутая работа с данными:**
 - NoSQL базы данных: Redis (для кэширования), MongoDB.
 - Масштабирование: репликация, шардинг БД, брокеры сообщений (Kafka).
- **Автоматизация и DevOps:**
 - CI/CD пайплайны: автоматическая сборка и деплой кода через GitHub Actions или GitLab CI.
 - Оркестрация: основы работы с Kubernetes.

- *Документирование*: составление интерактивных спецификаций API в OpenAPI/Swagger.
- **Командные и Бизнес-навыки (Soft Skills):**
 - *Методологии*: Scrum и Kanban, работа с task-трекерами (Jira, Kaiten).
 - *Аналитика*: сбор и формализация требований (написание ТЗ), чтение диаграмм UML/BPMN.
 - Участие в *Code Review* (совместный аудит чужого кода).
- **Практический проект:** Выпускной командный проект: микросервисное приложение, развернутое на удаленном сервере с настроенным CI/CD пайплайном, покрытое тестами, с полной документацией API в Swagger и ведением задач по Scrum.